Spring25 (A)

1. a)

i) Let's consider a scenario where a total of 4 processes are available for CPU allocation. Now, they have burst times like 4, 5, 6, 8. The OS is configured to use Round-Robin scheduling with a time quantum of 10. At this point, figure out the problem with this amount of time quantum.

-The time quantum (10 ms) is larger than all process burst times, so each process completes in a single turn without preemption. As a result, Round Robin behaves like FCFS scheduling. This removes the advantages of RR such as better responsiveness and fairness, making the time quantum inefficiently large.

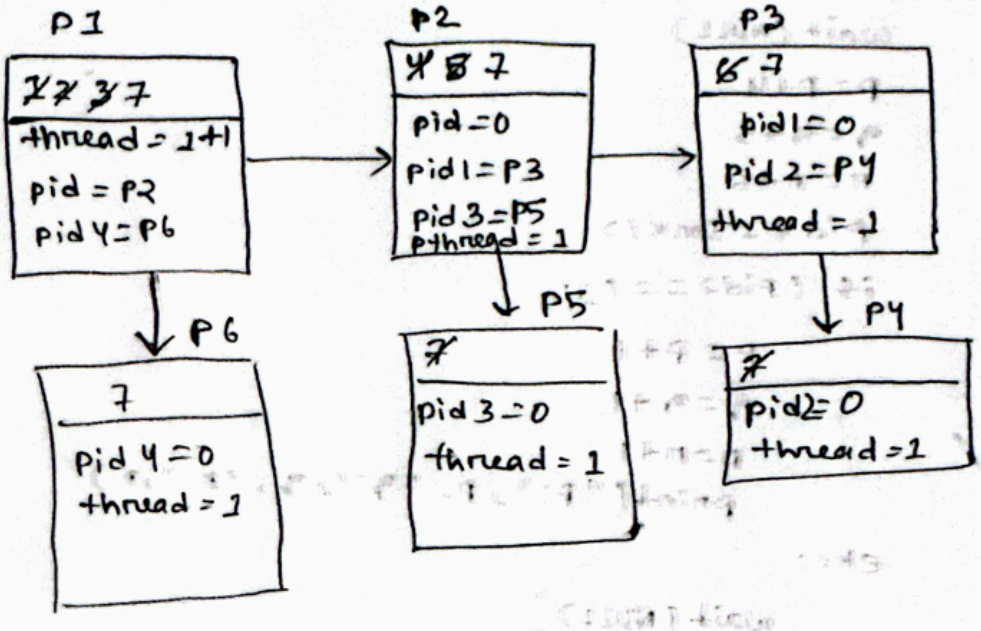
ii) Observe the following code. **Calculate** how many processes and how many threads will be created.

```
pid_t pid;
pthread_create( . . .);
pid = fork();
if (pid == 0) { /* child process */
      fork();
}
fork();
```

TOPIC NAME: Spring 25 (A) DAY: TIME: DATE: / /

① ii)

```
1 pid-t pid;
2 pthread_create(...);
3 pid = fork();
4 if (pid==0) { /* child process*/
5       fork();
6 }
7 fork()
```



Thread = 7

process = 6

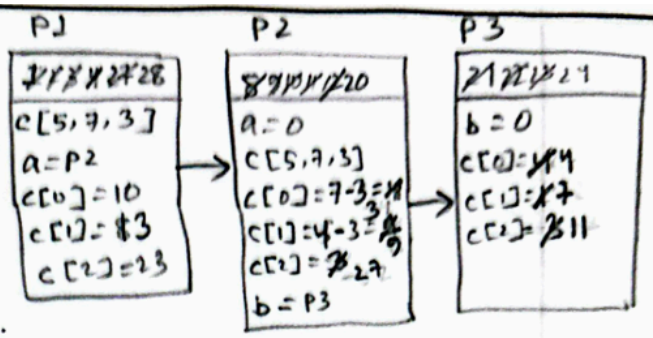
b) Find outputs of the given code: **[10* 0.5= 5]**

```
int main(){
    pid_t a,b;
    int c[]={5,7,3};
    a=fork();
    if(a<0){
        printf("error\n");
    }
    else if(a==0){
        c[0]=c[1]-c[2];
        c[1]=c[0]-c[2];
        c[2]=c[0]-c[1];
        b=fork();
        if(b<0){

        }
        else if(b>0){
            wait();
            c[0]=c[1]*c[2];
            c[1]=c[0]*c[2];
            c[2]=c[0]*c[1];
        }
        else{
            c[0]=c[1]+c[2];
            c[1]=c[0]+c[2];
            c[2]=c[0]+c[1];
        }
    }
    else{
        printf("hello\n");
        wait();
        c[0]=c[1]+c[2];
        c[1]=c[0]+c[2];
        c[2]=c[0]+c[1];
    }
    for(int i=0;i<3;i++){
        printf("c[%d]= %d\n",i,c[i]);
    }
    return 0;
}
```

b)

```
1  int main(){
2      pid_t a, b;
3      int c[] = {5, 7, 3};
4      a = fork();
5      if (a<0){
6          printf("error\n");
7      }
8      else if (a==0){
9          c[0] = c[1] - c[2];
10         c[1] = c[0] - c[2];
11         c[2] = c[0] - c[1];
12         b = fork();
13         if (b<0){
14         }
15         else if (b>0){
16             wait();
17             c[0] = c[1] + c[2];
18             c[1] = c[0] + c[2];
19             c[2] = c[0] + c[1];
20         }
21         else {
22             c[0] = c[1] + c[2];
23             c[1] = c[0] + c[2];
24             c[2] = c[0] + c[1];
25         }
26     }
27     else {
28         printf("hello\n");
29         wait();
30         c[0] = c[1] + c[2];
31         c[1] = c[0] + c[2];
32         c[2] = c[0] + c[1];
33     }
34     for (int i=0; i<3; i++){
35         printf("c[%d] = %d\n", i,
                   c[i]);
36     }
37     return 0;
   }
```



Output:

hello → P1

c[0] = 10
c[1] = 13
c[2] = 23 } P1

c[0] = 3
c[1] = 9
c[2] = 27 } P2

c[0] = 4
c[1] = 7
c[2] = 11 } P3

c) Write the answers in the answer script.

Dipu is implementing his own OS. For his OS he has decided to develop **Multilevel Feedback Queue** as a scheduler. To design the scheduler he decided to divide the ready queue into **three** levels. According to his preferences, in every level round-robin algorithm will be used and some constraints should be maintained in each level which are given below.

- **Q1:** time quantum = **3**, priority = **1**
- **Q2:** time quantum = **5**, priority = **5**
- **Q3:** time quantum = **10**, priority = **7**

Here, the lower priority value indicates the higher priority and in case of an I/O request, the scheduler will promote a process by one level upon I/O operation completion. At a certain moment, Dipu has collected information on burst times, arrival times, and I/O times of four processes

Process	Burst Time	Arrival Time	I/O Time
P1	19	0	6 [After **15s** of total CPU allocation]
P2	8	2	N/A
P3	9	12	4 [After **7s** of total CPU allocation]
P4	17	7	N/A

I. **Draw** a Gantt chart using multilevel feedback queue scheduling algorithm showing the states of the ready queues of different levels. **[12]**

II. **Calculate** the average waiting time and average turnaround time from the Gantt chart. **[1.5+ 1.5= 3]**

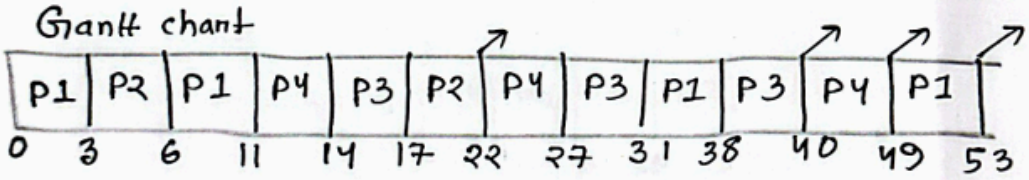
Spring 25 (A)

(1) c) (i)

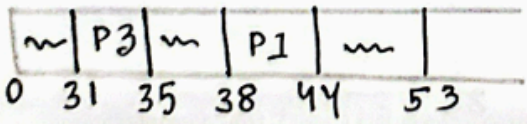
Process	Arrival time	Burst time	I/O time
P1	0	15 11 4 10 15 12 7 0	6 (after 15 3 CPU)
P2	2	8 5 0	N/A
P3	12	8 6 2 0 7 4 0	4 (after 7 s CPU)
P4	7	17 14 9 0	N/A

Q_1 (RR : 3)	Q_2 (RR : 5)	Q_3 (RR : 10)	I/O
0 : P1	3 : P1	11 : P1	31 : P3
2 : P2	6 : P1, P2	14 : P1	35 : { }
7 : P4	11 : P2	22 : P1, P3	38 : P1
12 : P3	14 : P2, P4	27 : P1, P4	44 : { }
17 : { }	17 : P2, P4, P3	31 : P1, P4	
35 : P3	22 : P4, P3	38 : P4	
44 : P3 P1	27 : P3	49 : { }	
53 : { }	31 : { }		

Gantt chart



I/O Gantt chart



(ii)

Process	Turnaround time	Waiting time
P1	53 − 0 = 53	53 − 19 − 6 = 28
P2	22 − 2 = 20	20 − 8 = 12
P3	40 − 12 = 28	28 − 9 − 4 = 15
P4	49 − 7 = 42	42 − 17 = 25
	average = 35.75	average = 20

2.a) i) You are given an array of 100 integers. You are required to calculate the sum of these integers using a 10-core CPU. Identify what kind of parallelism you need to implement to ensure that the operation is performed most efficiently.

To compute the sum of 100 integers using a 10-core CPU most efficiently, we should use: **Data Parallelism**

- **Why Data Parallelism?**
  - The task (summing integers) is the **same operation applied to different parts of data**.
  - We can divide the array into **10 equal chunks** (each with 10 integers).
  - Each core processes one chunk **simultaneously**.
- **How it works:**
  1. Split array → 10 parts

2. Each core computes a **partial sum**
3. Combine all partial sums → final result

🔹 **Example:**

- Core 1 → sum of elements 1–10
- Core 2 → sum of elements 11–20
- …
- Core 10 → sum of elements 91–100
- Final sum = sum of all 10 partial sums

🔹 **Why this is most efficient:**

- Maximizes CPU core utilization
- Minimizes idle time
- Very low dependency between tasks

ii) The BRACU CSE Department has unveiled its newly upgraded research lab featuring 55 high-performance PCs. These PCs are allocated to thesis groups based on their area of research. The allocation is as follows:

● 20 PCs are dedicated to AI/ML-related thesis groups.

● 18 PCs are dedicated to NLP-related thesis groups.

● The remaining PCs are assigned to Image Processing-related thesis groups.

Each thesis group is allocated some PC within their respective topic area for their allocated time. However, since the total number of thesis groups exceeds the number of available PCs, groups must wait their turn if all PCs are currently in use. Here, a strict constraint is maintained that each group can only use the PCs allocated to their area of research. During the thesis defense week, 23 NLP groups may try to access PCs, but there may not be enough PCs in the NLP section, and some of them may need to wait for their turn. **State which synchronization method can be used here to ensure the synchronization.**

The appropriate synchronization method here is **Counting Semaphore**.

Each research area (AI/ML, NLP, Image Processing) can have its own **semaphore initialized to the number of PCs**:

- AI/ML → 20
- NLP → 18
- Image Processing → 17

When a thesis group wants to use a PC, it performs **wait()** on its area's semaphore:

- If a PC is available → access is granted
- If all PCs are busy → the group **waits (blocked)**

After finishing, the group performs **signal()**, releasing the PC for others.

This works perfectly because:

- It controls **limited shared resources** (PCs)
- Ensures **no more than available PCs are used at once**
- Automatically handles **waiting and waking up groups**

Conclusion: Use **Counting Semaphores** for proper synchronization.

iii) You are booking a train ticket for your journey back home during Eid. When you visit the booking website, you find that there is only one seat left. You proceed to book it, and the booking is confirmed. The code used to book a seat is as follows:

```
lock = false
book_seat(){
    while(test_and_set(&lock));
      if (available_seats > 0) {    // Check if seats are
available
          available_seats = available_seats - 1  // Deduct
seat
        print("Booking confirmed!")  // Confirm booking
    }
    else {
        print("No seats available.")
    }
    lock = true;
}
```

Figure out the error from the above code.

The error is in the **lock release statement**.

Problem:

lock = true; (last lock)

This is incorrect because:

- test_and_set(&lock) sets lock = true when a process enters the critical section.
- To **release the lock**, it should be set back to **false**, not true.

Correct version:

lock = false;

🔹 **Why this is wrong:**

- If lock remains **true**, no other process can enter the critical section.
- This leads to **infinite waiting (deadlock/starvation)** for other users trying to book.

Conclusion:

The lock is **not properly released**. It should be set to **false after booking**, not true.

Code:

```
lock = false; // Initial state: Unlocked

book_seat() {
   while(test_and_set(&lock)); // Entry Section: Acquire lock

   if (available_seats > 0) {
      available_seats = available_seats - 1;
      print("Booking confirmed!");
   }
   else {
      print("No seats available.");
   }

   lock = false; // EXIT SECTION: Must set to false to release the lock
}
```

Solved by Azmari Sultana